UNIVERSIDADE DE SÃO PAULO
Instituto de Ciências Matemáticas e de Computação
Departamento de Ciências de Computação

Mídia interativa que auxilia o ensino de Sistemas Evolutivos aplicados à Robótica

Luca Gomes Urssi

São Carlos – SP

Mídia interativa que auxilia o ensino de Sistemas Evolutivos aplicados à Robótica

Luca Gomes Urssi

Orientador: **Prof. Dr. Eduardo do Valle Simões**

Monografia final de conclusão de curso apresentada ao Instituto de Ciências Matemáticas e de Computação – ICMC-USP, como requisito parcial para obtenção do título de Bacharel em Computação.
Área de Concentração: Sistemas Evolutivos

USP – São Carlos
Junho de 2024

Urssi, Luca Gomes
Mídia interativa que auxilia o ensino de Sistemas Evolutivos aplicados à Robótica / Luca Gomes Urssi. - São Carlos - SP, 2024.
47 p.; 29,7 cm.

Orientador: Eduardo do Valle Simões.
Monografia (Graduação) - Instituto de Ciências Matemáticas e de Computação (ICMC/USP), São Carlos - SP, 2024.

1. Sistemas Evolutivos. 2. OpenGL. I. Simões, Eduardo do Valle. II. Instituto de Ciências Matemáticas e de Computação (ICMC/USP). III. Título.

AGRADECIMENTOS

Agradeço primeiramente ao Professor Eduardo do Valle Simões, pela atenção e pelas ótimas orientações neste trabalho. Agradeço também ao Breno Cunha Queiroz, pelo seu tutorial em OpenGL que me inspirou a fazer este TCC.[1] E por ultimo agradeço a minha família e amigos pelo suporte e paciência ao longo dos anos.

[1] <https://github.com/Brenocq/OpenGL-Tutorial>

RESUMO

URSSI L. G.. **Mídia interativa que auxilia o ensino de Sistemas Evolutivos aplicados à Robótica**. 2024. 47 f. Monografia (Graduação) – Instituto de Ciências Matemáticas e de Computação (ICMC/USP), São Carlos – SP.

Este trabalho aborda os conceitos de sistemas evolutivos ensinados na disciplina de Sistemas Evolutivos aplicados à Robótica (SSC0713)(JUPITERWEB, 1999) do Instituto de Ciências Matemáticas e de Computação (ICMC) da Universidade de São Paulo (USP). Utiliza-se Open Graphics Library (OpenGL) para demonstrar de forma gráfica e interativa aos temas abordados.

O trabalho consiste em uma aplicação em C++ com OpenGL separada em fases que contém um ou mais temas de sistemas evoluídos por fase, incluindo os tópicos de iniciar uma população com número determinado de indivíduos e as características a serem simuladas, a seleção desses indivíduos pelo seu desempenho, a avaliação da população considerando as caracteristicas desejadas, a reprodução dos indivíduos de forma sexuada e assexuada, a aplicação de mutação e o conceito de genocidio para achar outros resultados.

Palavras-chave: Sistemas Evolutivos, OpenGL.

ABSTRACT

URSSI L. G.. **Mídia interativa que auxilia o ensino de Sistemas Evolutivos aplicados à Robótica**. 2024. 47 f. Monografia (Graduação) – Instituto de Ciências Matemáticas e de Computação (ICMC/USP), São Carlos – SP.

This project utilizes the theory of evolutionary systems from the course Sistemas Evolutivos aplicados à Robótica (SSC0713)(JUPITERWEB, 1999) from ICMC of USP. Using OpenGL to demonstrate in a visual and interactive way the topics covered.

The project is an an application in C++ with OpenGL sectioned in stages with one or more topics of evolutionary systems per stage, including the topics of initiating a population with the predetermined number of individuals and their caracteristics to be simulated, the selection of these individuals by their performance, the fitness of the population considering the desired caracteristics, the reproduction of the population using crossover and asexual reproduction, the application of mutation and the concept of genocide to find different results.

Key-words: evolutionary systems, OpenGL.

LISTA DE ILUSTRAÇÕES

Figura 1 – Diagrama simplificado de um sistema evolutivo 19
Figura 2 – Gráfico de ótimo local e global 21
Figura 3 – Bixinho desenhado em OpenGL 23
Figura 4 – Fase final com todo conteúdo. 25
Figura 5 – Menu principal 33
Figura 6 – Iniciar a População 35
Figura 7 – Elemento de menu : tipo de gene 36
Figura 8 – Iniciar População : resultados 36
Figura 9 – Seleção 38
Figura 10 – Seleção : Cor 38
Figura 11 – Seleção : Cor & Forma 39
Figura 12 – Elemento de menu : Reprodução 40
Figura 13 – Reprodução asexual, resultados 41
Figura 14 – Reprodução sexuada, resultados 42

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Função de criar retângulo 22
Código-fonte 2 – RenderString 27
Código-fonte 3 – Estrutura de Bixinho 28
Código-fonte 4 – Desenhar Bixinho 29
Código-fonte 5 – int main 30
Código-fonte 6 – void interface 31
Código-fonte 7 – void timer 31
Código-fonte 8 – void button_click 31
Código-fonte 9 – Exemplo de botão do menu principal 33
Código-fonte 10 – void Repopulate 40

LISTA DE ABREVIATURAS E SIGLAS

CUDA ... Compute Unified Device Architecture
ICMC .... Instituto de Ciências Matemáticas e de Computação
OpenGL .. Open Graphics Library
USP ..... Universidade de São Paulo

SUMÁRIO

1 INTRODUÇÃO 17
1.1 Motivação e Contextualização 17
1.2 Objetivos 18
1.3 Organização da Monografia 18

2 FUNDAMENTAÇÃO TEÓRICA 19
2.1 Sistemas Evolutivos 19
2.2 OpenGL 22

3 IMPLEMENTAÇÃO 25
3.1 Drawings.h 27
3.2 Bixinho.h 28
3.3 Estruturação 30
3.4 Main Menu 33
3.5 Iniciar a População 35
3.6 Seleção 38
3.7 Reprodução 40
3.8 Mutação 43
3.9 Genocídio 44

4 CONCLUSÃO 45

REFERÊNCIAS 47

Capítulo 1

INTRODUÇÃO

1.1 Motivação e Contextualização

Sistemas Evolutivos vem sendo cada vez mais aplicados na resolução de sistemas complexos, para os quais o acesso a especialistas é difícil. Alguns exemplos de aplicações de Sistemas Evolutivos seriam na área de modelagem matemática e computacional (AMORMINO; JÚNIOR, 2020) e na área de automação em ambientes industriais (CAVALCANTE, 2012). Com isso, a demanda por profissionais que dominem esse tema vem crescendo na indústria.

Para um aluno do ICMC da USP aprender esse conteúdo, é necessário participar da disciplina de Sistemas Evolutivos aplicados à Robótica (SSC0713)(JUPITERWEB, 1999), uma matéria optativa do curso de Engenharia da Computação, na qual é ensinado os fundamentos de Sistemas Evolutivos e como aplicá-los em robótica.

Dado a natureza da disciplina de simular a animação de vários objetos ao mesmo tempo e de forma rápida, é comum que se recomende aos alunos o uso de OpenGL(WOO *et al.*, 1999), uma biblioteca disponível em C++, python e outras linguagens, pois com ela e com a linguagem de processamento paralelo Compute Unified Device Architecture (CUDA)(NVIDIA; VINGELMANN; FITZEK, 2020) é possível processar e visualizar um grande número de objetos ao mesmo tempo. Isso se dá pelo fato do OpenGL ser rápido para desenhar na tela e que CUDA consegue utilizar a placa de vídeo para fazer o processamento em vez de utilizar o número limitado de núcleos do computador.

Apesar das vantagens de utilizar OpenGL, muitos alunos optam por não utilizar a ferramenta. Isto acontece porque, apesar de ser bem documentada, essa ferramenta é trabalhosa de utilizar.

1.2 Objetivos

O objetivo deste trabalho é implementar uma mídia visual e interativa, que utilize os conteúdos abordados na disciplina de Sistemas Evolutivos aplicados à Robótica (SSC0713)(JUPITERWEB, 1999), e OpenGL para que os alunos tenham mais exemplos prontos de código. Usando como base o tutorial feito por Breno Cunha Queiroz (QUEIROZ, 1999). Para isso, utilizou-se a linguagem C++ com a biblioteca OpenGL, em um ambiente Linux Mint.

1.3 Organização da Monografia

Este trabalho é dividido em 4 Capítulos:

- Este capítulo de introdução contextualiza este trabalho e o objetivo dele.
- O segundo capítulo introduz brevemente o conceito de sistemas evolutivos e como funciona a biblioteca OpenGL, ambos importantes para entender o conteúdo deste trabalho.
- No terceiro capítulo é explicado em detalhe a ferramenta de software desenvolvida neste trabalho.
- O quarto capítulo contém a conclusão.

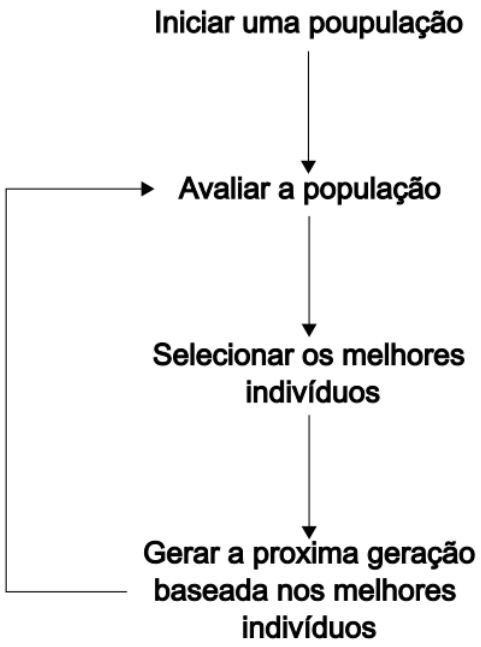
Capítulo 2

FUNDAMENTAÇÃO TEÓRICA

2.1 Sistemas Evolutivos

Neste sub capítulo será descrito o conteúdo de Sistemas Evolutivos(GABRIEL; DELBEM, 2008) ensinado na disciplina SSC0713 (JUPITERWEB, 1999) que é relevante para este trabalho. Um sistema evolutivo pode ser resumido de uma forma simplificada demonstrada na Figura 1.

Figura 1 – Diagrama simplificado de um sistema evolutivo



Fonte: criação do autor

Na Figura 1 o primeiro passo é *Iniciar uma população*, que significa escolher o que será simulado. Ao iniciar a população é necessário pensar no número de indivíduos e nas características deles que serão simuladas. Neste trabalho foi escolhido simular bixinhos, que são um grupo de formas coloridas. Com isso, a população é um certo número de bixinhos com duas características relevantes: forma e cor.

O segundo passo é *Avaliar a população*. A avaliação depende das características e do objetivo da simulação. Um exemplo que será visto neste trabalho é contar o número de indivíduos

que têm um certo tipo de característica. Dessa forma, a simulação tem como objetivo aumentar o número de indivíduos com a característica desejada.

No terceiro passo, são selecionados os melhores indivíduos. Quantos indivíduos serão selecionados dependerá do objetivo da simulação e dos métodos utilizados. No exemplo do segundo passo, os melhores seriam aqueles que tivessem a característica desejada.

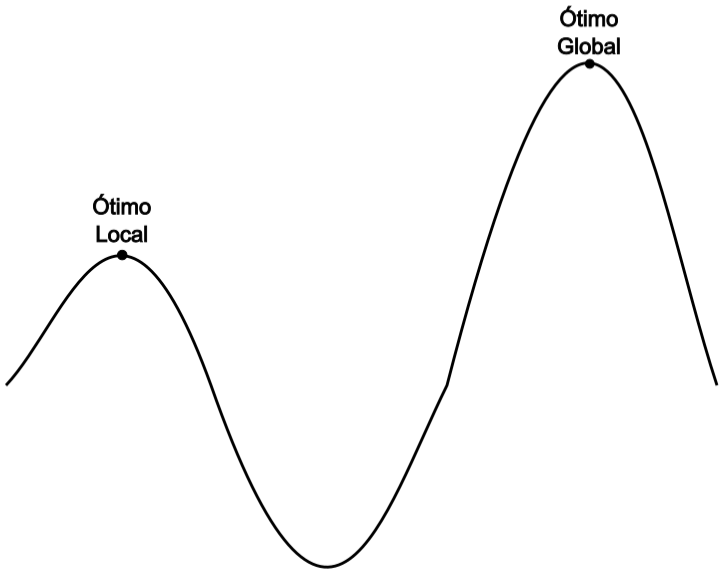
Por fim, o último passo trata de gerar a próxima geração, onde são utilizados os indivíduos selecionados no passo anterior. Em uma simulação onde os indivíduos se reproduzem por cruzamento para criar a próxima geração, o ideal é incentivar o cruzamento dos melhores indivíduos. Dessa forma, a geração seguinte tem uma influência positiva no passo da avaliação. Sempre que possível é bom manter o melhor indivíduo vivo em todas as simulações.

Ainda no último passo existe mais um conceito a ser considerado: a aplicação de mutação durante a criação da próxima geração. Mutação tem o papel de possibilitar que um indivíduo da geração seguinte possa ser melhor do que o melhor indivíduo atual.

Por exemplo, considerando-se números como indivíduos e que a avaliação busque o maior número, desta forma, ao passar para a próxima geração, os números serão uma média deles mesmos, individualmente, com o melhor atual. Passando gerações o suficiente, todos os números serão iguais ao melhor, mas não será possível passar do melhor. Mas se houver mutação entre cada geração, é possível que um dos números da próxima geração seja maior que o melhor indivíduo atual, melhorando a avaliação.

Em algumas simulações é possível que o melhor indivíduo não consiga ser o ótimo global, mesmo com o uso de mutação. Um método que pode ser utilizado para buscar o ótimo global, é o *Genocídio*. Nesse método o sistema pode ficar travado em um ponto, o qual é chamado de *ótimo local*. Em seguida todos os indivíduos são destruídos, como descrito no nome do método "Genocídio", mantendo o ótimo local e inciando uma nova população com características aleatórias. Essa nova população é avaliada normalmente junto com o ótimo local, iniciando-se um novo processo evolutivo na esperança que apareça outro indivíduo melhor.

Figura 2 – Gráfico de ótimo local e global



Fonte: criação própia

Na Figura 2 tem um exemplo de situação onde genocídio é necessário. Nessa figura os indivíduos são pontos na eixo x e que o objetivo seja achar o ponto x com o valor mais alto da linha. Imagine que o melhor indivíduo da primeira geração encontrou o ápice da linha no lado esquerdo da imagem ou um valor próximo dele. Com o passar das gerações, mesmo com mutação não é possível chegar no ápice da direita, que é o verdadeiro melhor de todos, isso acontece, porque em cada geração a mutação altera valores pequenos demais e esses valores não conseguem passar do meio da linha da Figura 2. A solução portanto é gerar novos indivíduos, possibilitando que um indivíduo seja criado próximo do ótimo global, se tornando o melhor dessa geração e continuando com a simulação.

2.2 OpenGL

OpenGL(WOO *et al.*, 1999) é uma biblioteca que possibilita fazer uma interface gráfica com baixo custo de processamento. Para visualizar como ela funciona pode-se imaginar duas matrizes, uma delas é a que o usuário consegue ver e a outra é utilizada para escrever, e será mostrado ao usuário futuramente. Ao passar um ciclo do *clock*, a primeira matriz é sobre-escrita pela outra, atualizando a tela a cada ciclo de *clock*. Um ciclo de *clock* é o tempo que leva para processar e desenhar na tela, nesse projeto há 30 ciclos por segundo. A eficiência está na escrita da tela, pois atualizar uma matriz diversas vezes e escrever na tela uma matriz apenas uma vez é mais rápido do que escrever diretamente na tela múltiplas vezes em um só ciclo de *clock*.

No Código-fonte 1, pode-se observar uma forma de desenhar um retângulo.

Código-fonte 1: Função de criar retângulo

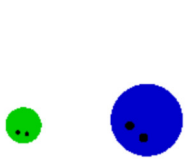
```
1 void retangle(float x, float y, float w, float h, float*color){
2   // Vai desenhar um polígono de 4 vértices
3
4   glColor3f(color[0], color[1], color[2]);
5   glBegin(GL_POLYGON); // Fala para o OpenGL que os próximos pontos
     serão para desenhar um polígono
6
7   // Adicionada cada vértice do retângulo
8   glVertex2d( x-w/2, y-h/2);
9   glVertex2d( x-w/2, y+h/2);
10  glVertex2d( x+w/2, y+h/2);
11  glVertex2d( x+w/2, y-h/2);
12
13  glEnd();// Fala para o OpenGL que terminou de enviar os vértices do
      polígono
14 }
```

Primeiramente escolher a cor do retângulo com a função *glColor3f*, que recebe 3 valores *float* de entrada, os quais são valores entre 0.0 e 1.0. Em seguida informar ao OpenGL que será iniciado um polígono, utilizando a função *glBegin(GL_POLYGON)* . Iniciado o polígono, imformar os pontos de vértice do retângulo, os quais nesta função são calculados baseados em um "x"e um "y"recebidos na entrada como o centro do retângulo e com a altura e largura como "h"e "w", também recebidos na entrada. Por fim anunciar *glEnd* que indicará ter terminado o polígono e sinalizar ao OpenGL para preencher o espaço dentro dos vértices com a cor escolhida.

De forma semelhante consegue-se desenhar um triângulo definindo apenas 3 vértices em vez de 4, ou pode-se desenhar um círculo definindo 360 vértices ao redor de uma posição central. Essas 3 funções mencionadas fazem parte do tutorial de OpenGL (QUEIROZ, 1999) criado por Breno Cunha Queiroz. Neste tutorial é ensinado como desenhar na tela e como mover

os desenhos na tela.

Figura 3 – Bixinho desenhado em OpenGL



Fonte: Recorte de imagem de exemplo (QUEIROZ, 1999)

Representado na Figura 3 acima, há dois bixinhos, que se referem a uma forma com dois olhos pretos que apontam para onde o bixinho está se movendo. Esses bixinhos fazem parte do tutorial e serão utilizados para demonstrar os conceitos durante esse trabalho.

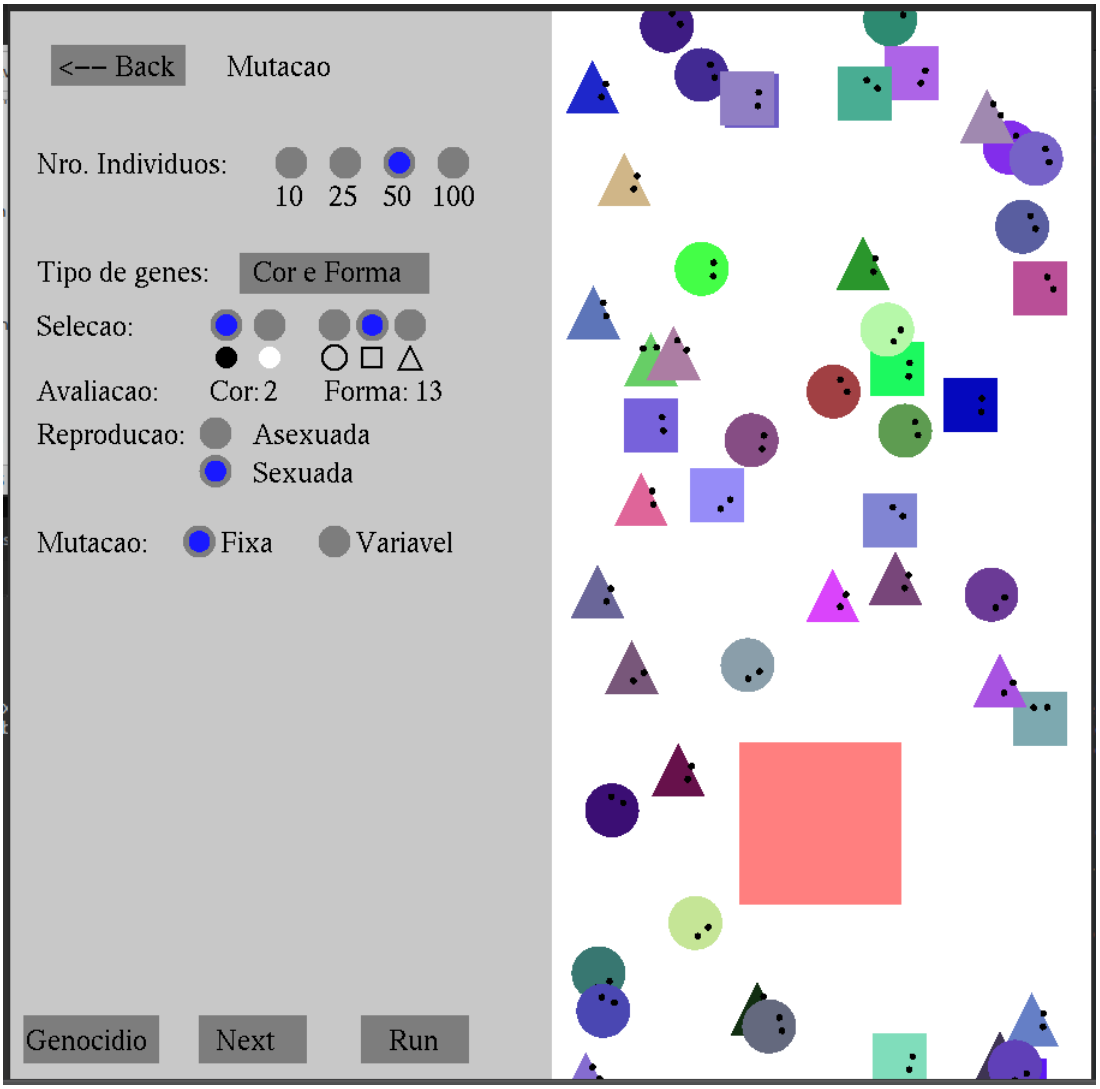
Capítulo 3

IMPLEMENTAÇÃO

Todo o software desenvolvido neste trabalho está disponivel como software livre sob GPL no repositório do github.[1]

Na Figura 4 pode ser vista a forma final da ferramenta de software proposta neste trabalho, com todos elementos de menu à esquerda e com os bixinhos sendo simulados à direita.

Figura 4 – Fase final com todo conteúdo.



Fonte: Foto da fase final deste trabalho

[1] <https://github.com/lucaurssi/TCC-Sistemas-Evolutivos>

A primeira tarefa feita neste trabalho foi utilizar os códigos prontos de exemplo em OpenGL (QUEIROZ, 1999) e enviá-los para uma biblioteca, separando-os em duas bibliotecas uma de desenhar na tela chamada *Drawings.h* e outra biblioteca específica para mexer com os bixinhos, chamada *Bixinho.h*.

3.1 Drawings.h

Nesta biblioteca foi adicionada às funções de desenhar retângulos, triângulos, círculos e outros polígonos. Essas funções recebem como entrada a posição central da forma e o(s) tamanho(s) relevante(s) das formas, recebem também um vetor de 3 posições para as cores.

Foram criadas duas funções essenciais para a produção deste trabalho; a função *setColor* e a função *RenderString*. A função *setColor* serve para passar os valores das cores vermelho, verde e azul, para uma variável de cor que é utilizada nas funções de desenhar na tela.

A função *RenderString* tem o papel importante de desenhar texto na tela, o código-fonte 2 abaixo representa essa função.

Código-fonte 2: RenderString

```
1 void RenderString(float x, float y, string string){
2
3   glColor3ub(0, 0, 0); // black
4   glRasterPos2f(x, y);
5   glutBitmapCharacter(GLUT_BITMAP_TIMES_ROMAN_24, string[0]);
6
7   for(int i=1; i<(int)string.length(); i++){
8     x = (x*900)+ 2*glutBitmapWidth(FONT, string[i-1]);
9     x = x/900;
10    glRasterPos2f(x, y);
11    glutBitmapCharacter(FONT, string[i]);
12  }
13 }
```

O código-fonte 2 tem como entrada o texto a ser escrito e a posição "x"e "y"com valores entre -1"e "1". Nesta função é utilizado *glColor3ub* para selecionar a cor preta, seguido de *glRasterPos2f* para escolher o local onde se vai escrever a primeira letra, por último *glutBitmapCharacter* que escreve a letra na fonte escolhida. Para escrever as outras letras é necessário alterar a posição de escrita, para fazer isso é adicionado a largura de cada letra na posição atual e repete-se o processo de escrita até acabarem as letras. O cálculo feito para achar a nova posição ocorre porque a largura está em pixels, mas "x"está entre -1"e "1". A largura da tela é de 900 pixels.

Nesta biblioteca também tem os desenhos dos elementos de menu que serão explicados nos seus respectivos subcapítulos.

3.2 Bixinho.h

Os bixinhos na figura 4 são representados no Código-fonte 3 como uma estrutura chamada de *Bixinho*. Essa estrutura e as funções de criar, desenhar e mover o bixinho foram baseadas no tutorial do Breno (QUEIROZ, 1999).

Código-fonte 3: Estrutura de Bixinho

```
1 #ifndef BIXINHO
2 #define BIXINHO
3
4 typedef struct _bixinho{
5   float radius;
6   float x;
7   float y;
8   float theta;
9   unsigned char r,g,b;
10  float vel;
11  unsigned char shape;
12 }Bixinho;
13
14 #endif
```

Nessa estrutura está sendo armazenado *radious* que é o tamanho dos bixinhos, a posição em x e y deles, *theta* que é a direção na qual se movem, sua cor em vermelho, verde e azul, sua velocidade e sua forma.

O tipo das variáveis de cor foram alterados de *float* para *unsigned char* por conta do custo de tempo em acessar a memória, na qual *unsigned char* é bem mais rápido, também foram considerados os valores armazenados em si, pois *unsigned char* armazena de 0 a 255, o que simplifica para trabalhar com cores. Outra alteração foi a adição de *shape* que armazena a forma de cada *bixinho*, pois na estrutura original todos os *bixinhos* eram círculos e não era necessário armazenar essa informação.

A função de criar um *bixinho* recebe a posição *x* e *y* dele e a *cor*, preenchendo os demais valores do *bixinho* com valores fixos, exceto *theta* e *shape* que são aleatórios dentre os possíveis valores.

A função de mover o *bixinho* muda ligeiramente o *theta* de forma aleatória e altera a posição *x* e *y* baseada no *theta*, na *velocidade* e também, se passou da borda da tela.

Por fim, na função de desenhar o *bixinho* no Código-fonte 4 foi adicionado uma condicional no lugar de desenhar o corpo do *bixinho*. Se a forma do *bixinho* não for relevante à simulação, o *bixinho* é desenhado com triângulos, diminuindo assim o custo de desenhar cada *bixinho*, caso contrário escolhe-se a forma dele baseado na variável *shape*. Os olhos do *bixinho*

que seguem a direção de *theta* não foram alterados.

Código-fonte 4: Desenhar Bixinho

```
1   //----- Desenha corpo do bixinho -----//
2
3   if (gene == 1) triangle(x, y, radius*2, radius*2, color);
4   else switch (shape){
5       case 0: circle(x, y, radius, color);
6         break;
7       case 1: retangle(x, y, radius*2, radius*2, color);
8         break;
9       case 2: triangle(x, y, radius*2, radius*2, color);
10        break;
11    }
```

3.3 Estruturação

Ao trabalhar com OpenGL foram utilizadas 3 funções principais:

- Função de desenhar na tela
- Função de processar objetos todo ciclo de *clock*
- Função de tratar *input* do usuário

Neste trabalho cada fase abaixo é uma biblioteca separada e contém uma versão destas 3 funções. Todo ciclo do *clock* chama a função de processar que calcula o movimento dos *bixinhos* presentes na tela e deleta, ou cria novos, conforme necessário. O menu só é desenhado uma vez ao entrar na fase e quando o usuário interage clicando na tela, isso foi uma decisão feita para economizar recursos não redesenhando a tela de menu se não houver necessidade.

Código-fonte 5: int main

```
1 int main(int argc, char** argv){
2     //----- Criar janela -----//
3
4     glutInit(&argc, argv);
5     glutInitDisplayMode(GLUT_RGB);
6     glutInitWindowSize(windowWidth, windowHeight);
7     glutInitWindowPosition(100, 100);
8     glutCreateWindow("TCC");
9     glClearColor(1.0, 1.0, 1.0, 1.0);// Limpa a tela
10    // --- 3 funcoes principais --- //
11
12    glutDisplayFunc(interface);// Define qual função irá desenhar
13
14    glutTimerFunc(0, timer, 0); // o processamento esta dentro da
      funcao timer
15
16  glutMouseFunc(button_click);// button_click eh chamada quando
      ocorre cliques na tela
17    // --------------------------- //
18
19    glutMainLoop();
20    return 0;
21 }
```

No código fonte 5 acima tem a função *main* do trabalho. Na função *main* pode-se observar que a função *interface* é chamada para desenhar na tela. No código fonte 6 a função de *interface* faz duas coisas: chamar outra função que corresponde a qual fase está sendo desenhada; e chamar a função *glutSwapBuffers* que troca a matriz buffer onde desenha com a matriz que

está na tela no momento. Em outras palavras, primeiro é preparado o que vai aparecer na tela e depois troca-se as matrizes efetivando o desenho.

Código-fonte 6: void interface

```
1 void interface(){
2   switch (menu_state){
3     case 0: main_menu();
4       break;
5     case 1: init_pop();
6       break;
7     case 2: Selecao();
8       break;
9     case 3: Reproducao();
10      break;
11    case 4: Mutacao();
12      break;
13    case 5: Genocidio();
14      break;
15    default: menu_state = 0;
16  }
17    glutSwapBuffers();
18  return;
19 }
```

No código fonte 5 a função de *interface* é chamada apenas uma vez. Para redesenhar a tela com frequência é necessário utilizar a função de *timer* representada no código fonte 7, onde *processInterface* de forma semelhante a *interface*, tem um *switch* para cada fase do trabalho. Depois de executar o que precisa ser processado, é chamada a função *glutPostRedisplay* que desenha novamente a função *interface*. Por fim, a função *timer* controla sua frequência com *glutTimerFunc* que chama *timer* recursivamente a cada 1 segundo dividido pelo número de *frames*.

Código-fonte 7: void timer

```
1 void timer(int){
2   processInterface(); // process
3   glutPostRedisplay(); // draw
4   glutTimerFunc(1000/30, timer, 0); // 30 fps
5 }
```

No código fonte 8 tem a função que é chamada toda vez que um botão é pressionado. Nesta função é tratado apenas o apertar do botão direito do mouse, que causa a chamada de outras funções que tratam do evento dependendo de onde o x e y do mouse está.

Código-fonte 8: void button_click

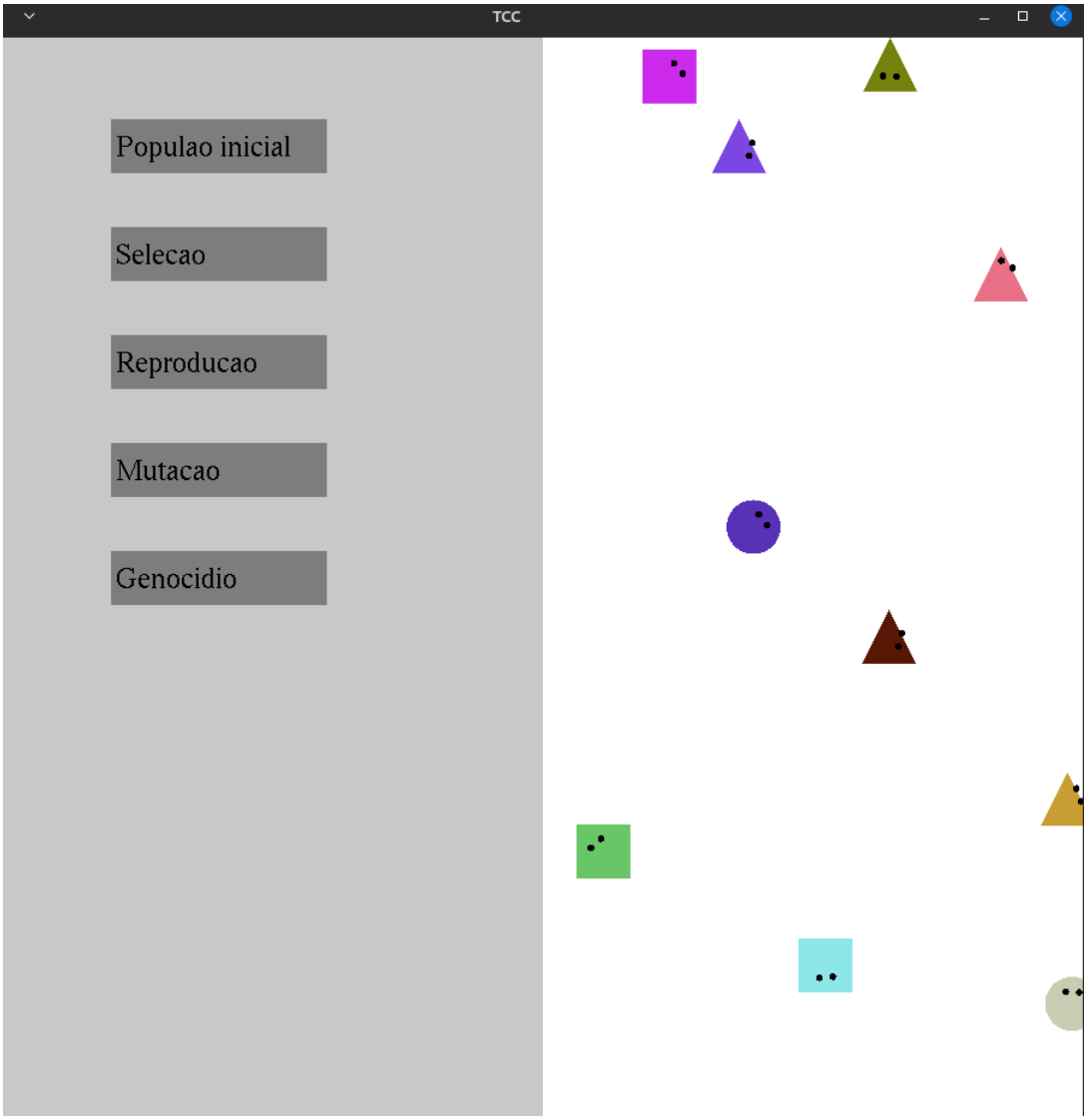
```
1 void button_click(int button, int state, int x, int y){
2   if(button==GLUT_LEFT_BUTTON && state==GLUT_DOWN)
3     switch (menu_state){
4       case 0: menu_buttons(x, y, &menu_state);
5         break;
6       case 1: IPop_buttons(x, y, &menu_state);
7         break;
8       case 2: Selecao_buttons(x, y, &menu_state);
9         break;
10      case 3: Reproducao_buttons(x, y, &menu_state);
11        break;
12      case 4: Mutacao_buttons(x, y, &menu_state);
13        break;
14      case 5: Genocidio_buttons(x, y, &menu_state);
15        break;
16    }
17  return;
18 }
```

Nos próximos sub-capítulos será explicado cada elemento de menu, comentando como foi desenhado na tela, seu propósito e comentários importantes.

3.4 Main Menu

O menu principal contém botões que levam o usuário a outras fases e bixinhos se mexendo apenas por estética.

Figura 5 – Menu principal



Fonte: Foto do menu principal do trabalho.

Os botões são compostos de um retângulo e uma palavra escrita com a função *RenderString*, no código fonte 9 abaixo é possível ver como foi desenhado um dos botões.

Código-fonte 9: Exemplo de botão do menu principal

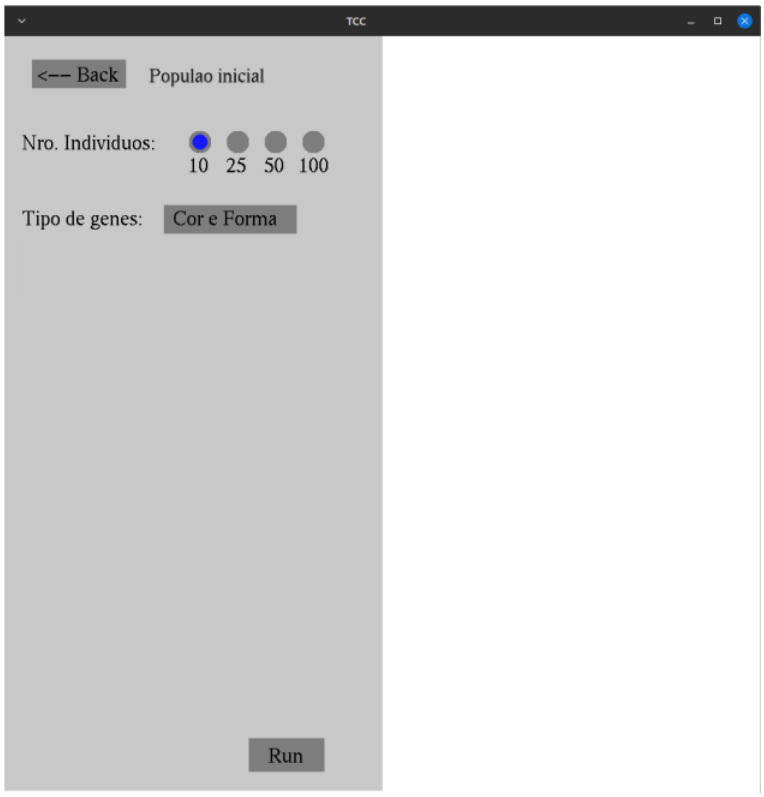
```
1     setColor(color, GREY); // GREY foi definido como '125,125,125'
2     retangle(-0.6, 0.8, 0.4, 0.1, color);
3     RenderString(-0.79, 0.78, "População inicial");
```

Ao apertar um dos botões, o programa chama a função do código fonte 8 que chama a função *menu_buttons*. Essa função trata das interações do usuário com o menu principal, que no caso só busca a posição do clique do mouse e checa se está em cima de um dos botões, caso esteja atualiza a variável *menu_state* para a fase que foi selecionada.

Os bixinhos do lado da tela são apenas ilustrativos no menu principal e não há como interagir com eles, eles apenas se movem de forma aleatória.

3.5 Iniciar a População

Figura 6 – Iniciar a População



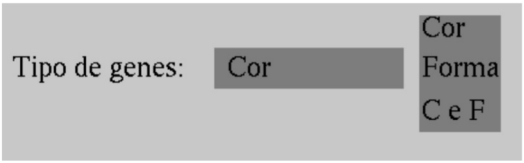
Fonte: Foto da interface da fase de iniciar população do trabalho.

Na Figura 6 pode observar a primeira fase do trabalho, onde escolhe-se o número de indivíduos a serem simulados e quais características serão consideradas. Na mesma imagem também existem dois botões, um de voltar ao menu principal e outro chamado *Run* que serve para iniciar a simulação.

O elemento de escolha do número de indivíduos define o número de bixinhos que vão ser desenhados na tela. Este elemento de menu foi criado utilizando a função *RenderString* do código fonte 2 e a função de criar círculo definida no mesmo subcapítulo. O círculo azul que representa a escolha atual funciona como um botão que define qual posição está ativa toda vez que esse elemento é interagido.

O segundo elemento trata dos genes que serão simuladas. Nesse elemento há 3 opções: cor, forma e cor & forma. Abaixo na Figura 7 observa-se o menu abrindo quando é clicado no quadrado cinza.

Figura 7 – Elemento de menu : tipo de gene



Fonte: Foto da interface da fase de iniciar população do trabalho, elemento de menu do tipo de gene.

Após selecionados o número de indivíduos e o tipo dos genes, clica-se em *Run* iniciando a simulação. Na Figura 8 há algumas combinações de escolha dos elementos de menu e seus resultados.

Figura 8 – Iniciar População : resultados



Fonte: Foto da interface da fase de iniciar população do trabalho, mostrando resultados.

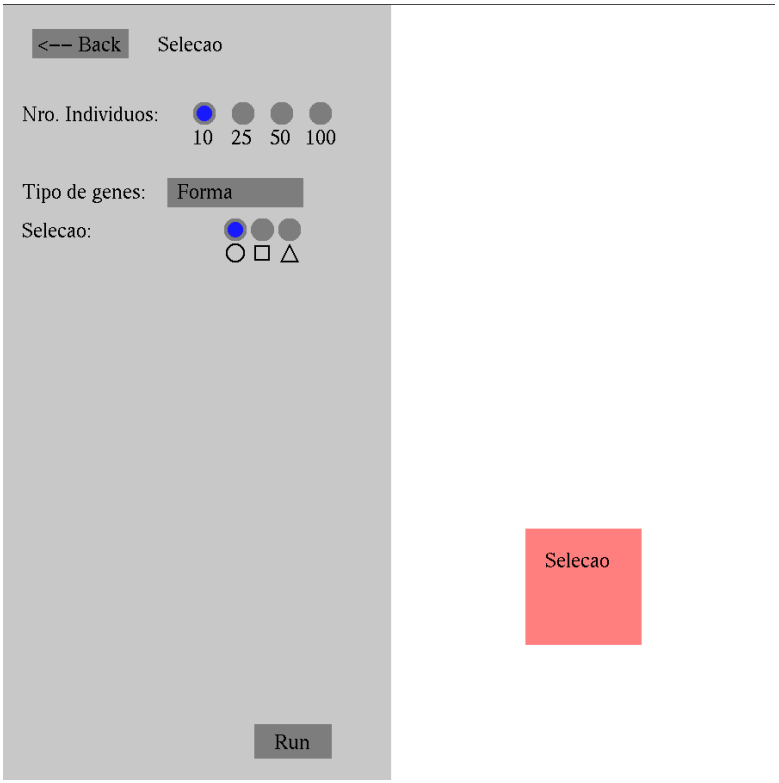
No primeiro resultado da Figura 8 há 10 indivíduos com o gene de cor, isso significa que 10 bixinhos serão desenhados com cores aleatórias e com a forma padrão de triângulo que é

a forma mais rápida de desenhar dentre as 3 formas do trabalho. No segundo resultado há 25 indivíduos com o gene de forma. Dessa forma 25 indivíduos sem cor foram desenhados com formas aleatórias entre triângulo, quadrado e círculo. No último resultado é demonstrado 100 indivíduos com cor e forma aleatória.

Uma última observação a ser feita na Figura 8 é o botão que aparece ao lado de *Run* chamado de *Next*. Este botão leva o usuário à próxima fase e só aparece após iniciar a simulação.

3.6 Seleção

Figura 9 – Seleção

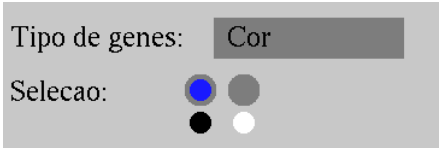


Fonte: Foto da interface da fase de Seleção.

Nesta fase permanecem os elementos da fase anterior e um novo elemento chamado *Seleção*. Esse elemento de menu funciona como o seletor de número de indivíduos. Ao selecionar o número de indivíduos e os genes relevantes, é escolhido dentre os genes quais são os mais desejados. Na Figura 9 foi escolhido o gene de formas, isso implica que será simulado 10 bixinhos sem cores com formas aleatórias.

A seleção ocorre no quadrado vermelho no lado direito inferior da tela. Quando um bixinho que não for da forma selecionada encostar no quadrado vermelho, ele é deletado, assim com o passar do tempo só restarão os bixinhos da forma que foi selecionada.

Figura 10 – Seleção : Cor



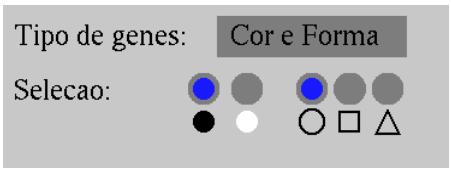
Fonte: Foto da interface da fase de Seleção, cor selecionado.

Na Figura 10 o gene indica as cores e que a seleção é o círculo preto, o que significa cores escuras. Com isso os bixinhos coloridos serão deletados até só sobrar os mais escuros. Essa seleção de claro e escuro é feita somando os 3 valores da cor do bixinho e checando se é

menor que 256 para escuro e maior que 512 para claro, pois os valores de vermelho, verde e azul vão de 0 a 255 por serem variáveis do tipo *unsigned char*.

Observando a Figura 9 e a Figura 10 observa-se que as escolhas de seleção só aparecem se o tipo de gene estiver presente, assim observando a Figura 11 pode-se ver ambos aparecendo.

Figura 11 – Seleção : Cor & Forma



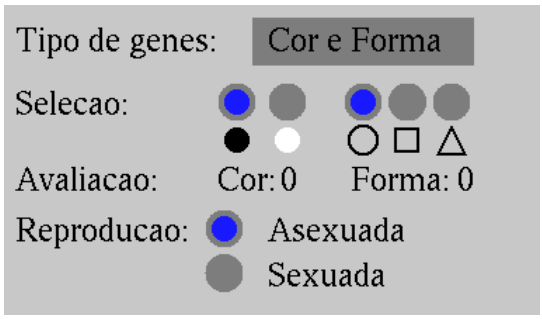
Fonte: Foto da interface da fase de Seleção, cor & forma selecionado.

Quando cor e forma estão selecionados, a seleção tenta chegar em bixinhos que tenham a forma desejada e a cor desejada. Por esta fase ter a seleção de dois genes, pode ocorrer que todos os bixinhos sejam deletados, pois são todos gerados aleatoriamente. Para observar melhor o que está acontecendo nesta fase ao utilizar esse simulador, é recomendado utilizar o tipo de gene de formas, pois este é mais fácil de visualizar.

3.7 Reprodução

Nesta fase são adicionados dois elementos de menu: *Avaliação* e *Reprodução*. Observando a Figura 12 nota-se que a Avaliação possui duas categorias: cor e forma, cada uma com um valor associado. Esses valores representam o número de bixinhos que possuem a cor ou forma correta respectivamente. Com esse elemento é possível saber o número de bixinhos que faltam ser selecionados.

Figura 12 – Elemento de menu : Reprodução



Fonte: Foto da interface da fase de Reprodução.

Na fase anterior, ao fazer a seleção, vários bixinhos são deletados sem serem repostos. Para repopular o número de indivíduos neste trabalho foi adicionado a forma dos bixinhos se reproduzirem. Neste trabalho serão utilizadas duas formas de reprodução: reprodução assexuada e sexuada.

Na reprodução assexuada, quando um indivíduo que possui a cor ou forma não selecionada passa pela área de seleção, ele é deletado. Quando isso ocorre imediatamente um novo indivíduo é criado. Esse novo indivíduo é uma cópia de outro indivíduo existente escolhido aleatoriamente.

Na reprodução sexuada, é escolhido 2 indivíduos aleatórios e o novo indivíduo criado recebe a média das cores dos indivíduos escolhidos, a posição x na tela de um e a posição y do outro, e por último um valor *theta* aleatório. No código fonte 10 observa-se a função que gera o novo indivíduo.

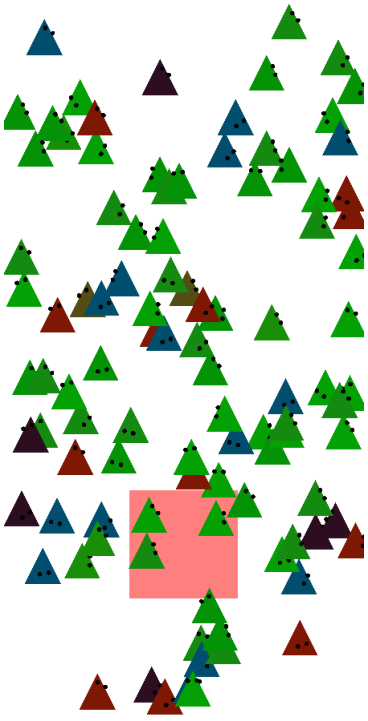
Código-fonte 10: void Repopulate

```
1 void Repopulate(){
2   char random_indv = rand()%Nro_individuos;
3   Bixinho A;
4
5     // Se for asexuado entra no if
6   if (SelecaoReproducao) A = Bc[random_indv];
7   else{
8     char random_indv2 = rand()%Nro_individuos;
9
```

```
10      A.radius = 0.05;
11      A.x = Bc[random_indv].x;
12      A.y = Bc[random_indv2].y;
13      A.theta = ((rand()%20)-10)/10.0;
14          A.vel = 0.010;
15
16          // cor
17      A.r = (Bc[random_indv].r + Bc[random_indv2].r)/2;
18      A.g = (Bc[random_indv].g + Bc[random_indv2].g)/2;
19      A.b = (Bc[random_indv].b + Bc[random_indv2].b)/2;
20
21          // forma de um ou do outro
22      if(rand()%2) A.shape = Bc[random_indv].shape;
23      else A.shape = Bc[random_indv2].shape;
24    }
25
26    Bc.push_back(A);
27    Nro_individuos++;
28  }
```

Na Figura 13 é observado o resultado da reprodução assexuada que gera 'clones' de outros bixinhos toda vez que um bixinho é deletado, até só sobrar apenas os bixinhos da cor e forma selecionados. No caso desta simulação foram selecionados os bixinhos mais escuros.

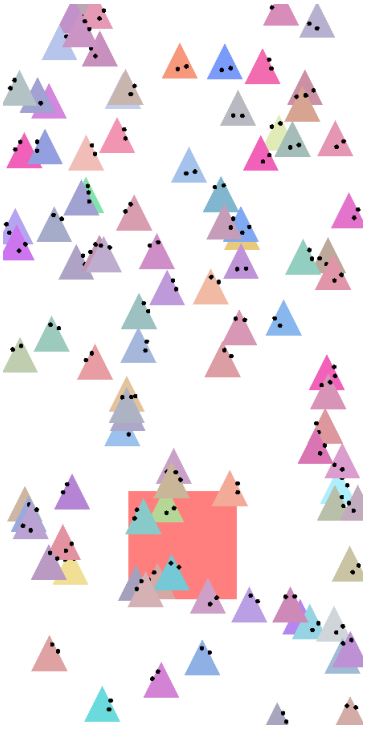
Figura 13 – Reprodução asexual, resultados



Fonte: Foto do resultado da fase de Reprodução com assexual selecionado.

Na Figura 14 observa-se o resultado da simulação utilizando reprodução sexuada e selecionando as cores claras. Os bixinhos da imagem são todos de cores claras e parecidas, pois após deletar cada bixinho não claro, há um cruzamento entre indivíduos aleatórios, até só sobrarem as cores claras.

Figura 14 – Reprodução sexuada, resultados



Fonte: Foto do resultado da fase de reprodução com sexuado selecionado.

3.8 Mutação

Nesta fase é explorado o conceito de mutação fixa e variável durante a reprodução. Neste trabalho a mutação altera a cor do bixinho criado por um valor fixo e a forma recebe uma pequena chance de mudar para uma forma diferente.

A cor do bixinho é representada por 3 valores de cor primária que vão de 0 a 255. Durante a reprodução é adicionado um valor entre -1 e 1 em cada cor primária alterando o valor final da cor do bixinho ligeiramente. Isso possibilita gerar novos bixinhos com a cor mais clara ou mais escura do que o melhor bixinho simulado até então.

Como a mutação é aleatória, há a possibilidade de surgir bixinhos piores do que os atuais. Por isso, é recomendado manter artificialmente o melhor bixinho da simulação. Nesta fase o melhor bixinho é encontrado e protegido de ser destruído até surgir um novo bixinho melhor que ele.

A forma dos bixinhos não tem como ser alterada ligeiramente como a cor. Portanto foi criado uma probabilidade de 5% de receber a mutação.

A mutação variada, se utilizada corretamente, agiliza a busca pelo melhor valor possível. Neste trabalho, quando é iniciado a população pode-se considerar que a mutação está em 100%, pois todos os valores são aletatórios dentro dos possíveis valores. O objetivo da mutação variada é chegar rápido em uma posição próxima do resultado ideal com a mutação alta e depois diminuir a mutação para se aproximar mais.

Na seleção de cor após gerados os indivíduos o valor da mutação é iniciado em -10 a 10. Em seguida, um contador começa a contar toda vez que um bixinho é deletado. Este contador é reiniciado toda vez que o novo bixinho é considerado melhor que o melhor atual. Se esse contador chegar em 20, reinicia o contador e diminui o valor da mutação para -5 a 5. Dessa forma diminuindo a aleatoriedade da mutação. Após outros 20 indivíduos não melhorarem, a mutação diminui novamente para -2 a 2 e depois para -1 a 1. Por fim, a mutação se mantém em -1 a 1.

A mutação variada com formas só busca ter um indivíduo na forma certa e após isso não serve como um bom exemplo para esta fase.

3.9 Genocídio

O conceito de genocídio surge em sistemas evolutivos quando uma população chega no que ela considera ser o melhor indivíduo e mesmo com mutação não encontra outro melhor, mas ainda é possível existir um indivíduo que é melhor do que o encontrado.

Com isso são destruídos todos os bixinhos mantendo o melhor, dessa forma gerando novos bixinhos aleatórios dentro dos possíveis que podem ser melhor que os encontrados até então.

Para demonstrar isso no trabalho, foi removida a mutação que sempre chega no melhor. Agora o melhor raramente chega na cor branca ou na cor preta. Foi também adicionado um botão visível na Figura 4 chamado *Genocídio* que mata todos os bixinhos, menos o melhor e gera novos bixinhos aleatórios, com a esperança que apareça um melhor.

Capítulo 4

CONCLUSÃO

A ferramenta de software criada para esse trabalho cumpriu com o objetivo proposto de utilizar OpenGL(WOO *et al.*, 1999) para criar uma interface gráfica e interativa. Foi também utilizado todo o conteúdo de sistemas evolutivos abordado na disciplina de Sistemas Evolutivos aplicados à Robótica (SSC0713)(JUPITERWEB, 1999).

Durante o desenvolvimento deste trabalho o aprendizado em OpenGL aprofundou-se bastante, pois foi necessário ler a documentação e utilizar funções diferentes das anteriores. Uma das funções diferentes foi a função que escolhe a cor, pois a função utilizada no tutorial do Breno (QUEIROZ, 1999) recebe como entrada variáveis *float*, mas foi trocado de *float* para *unsigned char* na implementação, pois a documentação sugeria utilizar o tipo correto na função. Nessa documentação foram encontradas diversas funções que só mudam o tipo da variável de entrada.

Uma das funções que necessitou utilizar a documentação de OpenGL foi a escrita de texto, onde a função encontrada escrevia apenas uma letra por vez e foi necessário criar uma função própria para escrever palavras, espaçando as letras.

Outro assunto que foi explorado durante este trabalho foram alguns conceitos de C++ e de *makefile*. No *makefile* utilizou-se o comando *wildcard* para compilar um número desconhecido de arquivos do mesmo tipo. Em C++ foram criados *headers* para conter cada tópico e tratar de problemas com declaração de variáveis repetidas dentro do *header*, quando ele é chamado em mútiplos outros *headers*.

Neste trabalho, em algumas das mecânicas e dos exemplos, foram buscadas formas diferentes das implementadas de como demonstrar o conteúdo. Como exemplo, quando é feita a seleção por cor, em vez de selecionar por claro ou escuro, foi pensado em selecionar pelas cores primárias ou por uma cor que o usuário venha decidir. Mas no final foi escolhido não alterar o trabalho dessa forma, pois não houve tempo suficiente para fazer tais alterações. Essas alterações pensadas e com a adição de novos exemplos de códigos utilizando OpenGL, poderam ser o início de um trabalho futuro.

REFERÊNCIAS

AMORMINO, G.; JÚNIOR, S. Método de imputação incremental, simples e eficiente para dados ausentes em sistemas evolutivos. 2020. Citado na página 17.

CAVALCANTE, A. L. D. Arquitetura baseada em agentes e auto-organizável para a manufatura. 2012. Citado na página 17.

GABRIEL, P. H. R.; DELBEM, A. C. B. **Fundamentos de algoritmos evolutivos**. [S.l.]: ICMC-USP, 2008. Citado na página 19.

JUPITERWEB. **Sistemas Evolutivos aplicados à Robótica**. 1999. Disponível em: <https://uspdigital.usp.br/jupiterweb/obterDisciplina?nomdis=&sgldis=ssc0713>. Citado 6 vezes nas páginas 5, 7, 17, 18, 19 e 45.

NVIDIA; VINGELMANN, P.; FITZEK, F. H. **CUDA, release: 10.2.89**. 2020. Disponível em: <https://developer.nvidia.com/cuda-toolkit>. Citado na página 17.

QUEIROZ, B. C. **OpenGL-Tutorial**. 1999. Disponível em: <https://github.com/Brenocq/OpenGL-Tutorial>. Citado 6 vezes nas páginas 18, 22, 23, 26, 28 e 45.

WOO, M.; NEIDER, J.; DAVIS, T.; SHREINER, D. **OpenGL programming guide: the official guide to learning OpenGL, version 1.2**. [S.l.]: Addison-Wesley Longman Publishing Co., Inc., 1999. Citado 3 vezes nas páginas 17, 22 e 45.